

SOLID

Contents

1. [The SOLID Principle](#)
2. [Single Responsibility Principle \(SRP\)](#)
3. [Open Close Principle \(OCP\)](#)
4. [Liskov Substitution Principle \(LSP\)](#)
5. [Interface Segregation Principle \(ISP\)](#)
6. [Dependency Inversion Principle \(DIP\)](#)

SOLID

SOLID is short for basic five principles of OOP, which was introduced in the early 2000s and adopted widely in the software industry. When these principles are combined together, a programmer can create an application that will be easy to maintain and can be extended over time.

The SOLID abbreviation is defined as follows:

- S: Single responsibility principle
- O: Open closed principle
- L: Liskov substitution principle
- I: Interface segregation principle
- D: Dependency inversion principle

[Back to Contents](#)

Single Responsibility

This states that *a class/method should have only one reason to change it, and this means, it should have a single job.*

You may have heard the quote: *“Do one thing and do it well”.*

Look at the example below and how it violates this principle.

```
public class AccountController {
    public void changeAccountOwnership(String accountId) {
        Account account = [SELECT Id, OwnerId FROM Account WHERE
Id = :accountId];
        account.OwnerId = UserInfo.getUserId();
        update account;
    }

    public void sendOwnerEmail() {
        //send email logic
    }
}
```

There are two violations of SRP, do you see them?

- *AccountController* is doing too much, it can change an owner and send emails.
- *changeAccountOwnership* is doing too much, it queries an account, it changes the owner and it does DML on an account.

Now look at the example below and see how we have corrected them.

```
public class AccountController {
    public AccountDataAccessorInterface accountDataAccessor = new
AccountDataAccessor();
    public void changeAccountOwnership(String accountId) {
        Account account =
accountDataAccessor.queryAccountById(accountId);
        account.OwnerId = UserInfo.getUserId();
        accountDataAccessor.updateAccount();
    }
}
```

```
public interface AccountDataAccessorInterface {
    Account queryAccountById(String accountId);
    void updateAccount(Account account);
}
```

```
public class AccountDataAccessor implements AccountDataAccessorInterface {
    public Account queryAccountById(String accountId) {
        //Query logic
    }

    public void updateAccount(Account account) {
        //update logic
    }
}
```

Can you see the difference? By creating the Data Accessor interface we have created a class/method pairing that handle all database interactions for Accounts.

The advantages of SRP are as follows: * It makes code as easy as possible to reuse * Small classes can be modified easily * Small classes are more readable

[Back to Contents](#)

Open Close

This states that *entities of software, such as classes, methods and functions, should be open for extension but closed for modification*. This means that classes and methods should be allowed to be extended without modification.

Any changes made in any existing code can potentially impact the entire system.

Look at the example below.

```
public class AreaCalculator {
    public Double calculateArea(List<Object> shapes) {
        Double area = 0;
        for (Object shape : shapes) {
            if (shape.type = Rectangle) {
                Rectangle rectangle = (Rectangle) shape;
                area += rectangle.Width *
rectangle.Height;
            } else {
                Circle circle = (Circle)shape;
                area += circle.Radius * circle.Radius *
Math.PI;
            }
        }
        return area;
    }
}
```

Now try adding a new shape to the calculation, do you see how this can get out of hand?

So lets follow OCP and allow our functionality to be extended.

```
public abstract class Shape {
    public abstract double Area();
}
```

```
public class Rectangle extends Shape {
    public Double Width {get; set;}
    public Double Height {get; set;}

    public override Double calculateArea() {
        return Width * Height;
    }
}
```

```
public class Circle extends Shape {
    public Double Radius {get; set;}
}
```

```
    public override Double calculateArea() {  
        return Radius * Radius * Math.PI;  
    }  
}
```

In this way we can continue to extend the functionality without affecting any existing functionality.

[Back to Contents](#)

Liskov Substitution

This will probably be the most difficult to understand This states *that if class B is a child of class A, then A can be replaced by B, without changing anything in a program*. In other words, the LSP principle states that you should not encounter unexpected results if child (derived) classes are used instead of parent classes.

Rectangle-Square Problem

```
public virtual class Bird {  
    public virtual void setLocation(double longitude, double latitude)  
{  
        //logic  
    }  
    public virtual void setAltitude(double altitude) {  
        //logic  
    }  
    public virtual void draw() {  
        //logic  
    }  
}
```

```
public class Penguin extends Bird {  
    public override void setAltitude(double altitude) {  
        ////altitude can't be set because penguins can't fly  
        //this method does nothing  
    }  
}
```

If an override method does nothing or just throws an exception, then you're probably violating the LSP.

The whole point of using an abstract base class is so that, in the future, you can write a new subclass and insert it into existing, working, tested code. This is the essence of the Open Closed Principle. However, when the subclasses don't adhere properly to the interface of the abstract base class, you have to go through the existing code and account for the special cases involving the delinquent subclasses. **This is a blatant violation of the Open Closed Principle.**

So how do we solve our problem?

```
public virtual class Bird {  
    public virtual void setLocation(double longitude, double latitude)  
{  
    //logic  
}  
    public virtual void draw() {  
    //logic  
}  
}
```

```
public virtual class FlightfulBird extends Bird {  
    public virtual void setAltitude(double altitude) {  
    //logic  
}  
}
```

```
public class Penguin extends Bird {  
    public override void setLocation(double altitude) {  
    //logic  
}  
  
    public override void draw(double altitude) {  
    //logic  
}  
}
```

This is but one simple example of how to correct a violation of this principle, however, this situation can manifest in a broad variety of ways, and is not always easy to identify.

[Back to Contents](#)

Interface Segregation

In programming, the interface segregation principle states that no client should be forced to depend on methods it does not use. Put more simply: Do not add additional functionality to an existing interface by adding new methods. Instead, create a new interface and let your class implement multiple interfaces if needed.

Look at the example below, can you see the problem?

```
public interface ProductInterface {  
    String getName();  
    String getAuthor();  
}
```

```
public class Movie Implements ProductInterface {
    private String movieName;

    public String getName() {
        return movieName;
    }

    public String getAuthor() {
        return new CustomException('Method not supported!');
    }
}
```

As we remember from LSP, if a method does nothing or just returns an exception you have a major problem.

So lets fix this.

```
public interface ProductInterface {
    String getName();
}
```

```
public interface BookInterface {
    String getAuthor();
}
```

```
public interface MovieInterface {
    String getDirector();
}
```

```
public class Books implements ProductInterface, BookInterface {
    private String bookName;
    private String author;

    public String getName() {
        return bookName;
    }

    public String getAuthor() {
        return author;
    }
}
```

Dependency Inversion

In programming, the dependency inversion principle is a way to decouple software modules. This principle states that * High-level modules should not depend on low-level modules. Both should depend on abstractions. * Abstractions should not depend on details. Details should depend on abstractions.

So lets look at a common implementation of this problem.

```
public interface ShapeInterface {  
    Integer calculateArea(Map<Integer, Integer> calcValues);  
}
```

```
public Rectangle implements ShapeInterface {  
    public Integer calculateArea(Map<Integer, Integer> calcValues) {  
        Integer area = 0;  
        //Logic  
        return area;  
    }  
}
```

```
public class CalculateRectangleArea {  
    public Integer calculateTheAreaOfRectangle(Map<Integer, Integer>  
calcValues) {  
        public ShapeInterface shapeRectangle = new  
Rectangle(); //oops this is now tightly coupled  
        return shapeRectangle.calculateArea(calcValues);  
    }  
}
```

As you can see in the above example, our code is now tightly coupled with "hard-coded" references to each other.

Lets take a look at how we can remove this dependency.

```
public interface ShapeInterface {  
    Integer calculateArea(Map<Integer, Integer> calcValues);  
}
```

```
public Rectangle implements ShapeInterface {  
    public Integer calculateArea(Map<Integer, Integer> calcValues) {  
        Integer area = 0;  
        //Logic  
        return area;  
    }  
}
```

```
    }  
}
```

```
public Circle implements ShapeInterface {  
    public Integer calculateArea(Map<Integer, Integer> calcValues) {  
        Integer area = 0;  
        //Logic  
        return area;  
    }  
}
```

```
public class CalculateRectangleArea {  
    public Integer calculateTheAreaOfRectangle(String className,  
Map<Integer, Integer> calcValues) {  
        Type t = Type.forName(className);  
        Shape iClass = (Shapes)t.newInstance();  
        return iClass.calculateArea(calcValues);  
    }  
}
```

By using dependency injection we no longer rely on the class to define the specific type of *Shape*.

[Back to Contents](#)